НИТУ «МИСИС»

Кафедра инженерной кибернетики

ОТЧЕТ

ПО

ЛАБОРАТОРНОЙ РАБОТЕ №7

«SIMD-оптимизация сверточных операторов обработки изображений на примере оператора Собела с использованием различных наборов команд»

учебная дисциплина «Параллельные вычисления»

Студент: Голубев И. М., БПМ-22-ПО-1

Преподаватель: Ватутин Э. И.

Оценка:

Дата защиты: 20.04.2026

2026 г.

Введение

Основная цель работы:

Познакомиться с принципами организации и программирования векторных расширений системы команд современных процессоров семейства x86.

Описание решаемой задачи

Задача в соответствии с **вариантом №7:** Реализовать оператор размытия порядка 3, используются маска размытия **M** (матрица 3x3, все элементы которой — единицы) и нормирующие множитель **d=1/9**. Решение необходимо выполнить в двух вариантах:

(a) Высокоуровневая (**скалярная**) реализация на C++ (последовательный расчёт свёртки для всех пикселей);

(b) **SIMD-оптимизированная** реализация с использованием наиболее нового векторного расширения, поддерживаемого процессором, с параллельной обработкой нескольких пикселей за одну инструкцию.

Корректность SIMD-версии должна быть подтверждена сопоставлением результатов со скалярной реализацией, а эффективность — измерением времени выполнения и оценкой ускорения.

Входные данные:

Изображение для обработки, `img.png` представляет из себя двумерную матрицу **800×800**, где каждый элемент $\mathbf{I_{xy}}$ — целое значение яркости пикселя в диапазоне **[0; 255]**. Изображение хранится в виде массива байтов `uint8_t` (файл `img.raw`), по одному байту на пиксель. Обработка выполняется для всех пикселей, за исключением граничных элементов матрицы:

Рисунок 1. Входное изображение `img.png` размера **800×800**

Выбранные средства для разработки

Для выполнения лабораторной работы использовались следующие программные средства и технологии:

- Язык программирования: C++17
- Среда разработки: Zed
- Операционная система: Fedora Linux
- Процессор: Intel(R) Core(TM) i5-10300H CPU @ 2.50GHz
- Размер RAM: 15Gi
- Компилятор: `g++`
- Компиляция и сборка программы:

  `g++ -O3 -mavx2 -std=c++17 lab07.cpp -o lab07`
- Запуск программы:

  `./lab07`

Листинг кода программы

Код доступен в удалённом Github репозитории по ссылке:

https://github.com/g0lubew-iv/parallel-comp/blob/main/lab07/lab07.cpp

```
#include <immintrin.h>
#include <cpuid.h>
#include <cstdint>
#include <fstream>
#include <iostream>
#include <vector>
#include <chrono>
#include <algorithm>
#include <cstdlib>

constexpr int W = 800;
constexpr int H = 800;
constexpr int N = W * H;

static uint8_t src[N];


static void blur3x3_scalar(const uint8_t* s, uint8_t* d, int w, int h)
{
    std::fill(d, d + w * h, 0);
    for (int y = 1; y < h - 1; ++y) {
        const uint8_t* r0 = s + (y - 1) * w;
        const uint8_t* r1 = s + (y    ) * w;
        const uint8_t* r2 = s + (y + 1) * w;

        uint8_t* out = d + y * w;
```

```
        for (int x = 1; x < w - 1; ++x) {
            int sum = r0[x - 1] + r0[x] + r0[x + 1] + r1[x - 1] + r1[x] + r1[x + 1]
+ r2[x - 1] + r2[x] + r2[x + 1];
            out[x] = (uint8_t)(sum / 9);
        }
    }
}


static inline __m128i div9_u16_to_u8_exact(__m256i s_u16) {
    const __m256i mul = _mm256_set1_epi16(7282);
    __m256i q_u16 = _mm256_mulhi_epu16(s_u16, mul);
    __m128i lo = _mm256_castsi256_si128(q_u16);
    __m128i hi = _mm256_extracti128_si256(q_u16, 1);
    return _mm_packus_epi16(lo, hi);
}


static void blur3x3_avx2(const uint8_t* s, uint8_t* d, int w, int h) {
    std::fill(d, d + w * h, 0);

    std::vector<uint16_t> H0(w), H1(w), H2(w);

    auto horiz_sum = [&](int y, std::vector<uint16_t>& Hr) {
        Hr[0] = Hr[w - 1] = 0;
        const uint8_t* r = s + y * w;

        int x = 1;
        for (; x + 15 < w - 1; x += 16) {
            __m128i a = _mm_loadu_si128((const __m128i*)(r + (x - 1)));
            __m128i b = _mm_loadu_si128((const __m128i*)(r + (x)));
            __m128i c = _mm_loadu_si128((const __m128i*)(r + (x + 1)));

            __m256i a16 = _mm256_cvtepu8_epi16(a);
            __m256i b16 = _mm256_cvtepu8_epi16(b);
            __m256i c16 = _mm256_cvtepu8_epi16(c);

            __m256i sum = _mm256_add_epi16(_mm256_add_epi16(a16, b16), c16);
            _mm256_storeu_si256((__m256i*)(Hr.data() + x), sum);
        }

        for (; x < w - 1; ++x) {
            Hr[x] = (uint16_t)r[x - 1] + (uint16_t)r[x] + (uint16_t)r[x + 1];
        }
    };

    horiz_sum(0, H0);
    horiz_sum(1, H1);

    for (int y = 1; y < h - 1; ++y) {
        horiz_sum(y + 1, H2);

        uint8_t* out = d + y * w;

        int x = 1;
        for (; x + 15 < w - 1; x += 16) {
            __m256i h0 = _mm256_loadu_si256((const __m256i*)(H0.data() + x));
            __m256i h1 = _mm256_loadu_si256((const __m256i*)(H1.data() + x));
            __m256i h2 = _mm256_loadu_si256((const __m256i*)(H2.data() + x));

            __m256i s3 = _mm256_add_epi16(_mm256_add_epi16(h0, h1), h2);
            __m128i out16 = div9_u16_to_u8_exact(s3);
```

```
            _mm_storeu_si128((__m128i*)(out + x), out16);
        }

        for (; x < w - 1; ++x) {
            uint16_t sum = (uint16_t)(H0[x] + H1[x] + H2[x]);
            out[x] = (uint8_t)(sum / 9);
        }

        H0.swap(H1);
        H1.swap(H2);
    }
}

static int max_abs_diff(const uint8_t* a, const uint8_t* b, int n) {
    int md = 0;
    for (int i = 0; i < n; ++i) {
        int d = std::abs((int)a[i] - (int)b[i]);
        md = std::max(md, d);
    }

    return md;
}


template <class F>
static double bench_ms(F&& f, int iters) {
    using clock = std::chrono::steady_clock;
    for (int i = 0; i < 5; ++i) {
        f(); // warm up
    }

    auto t0 = clock::now();
    for (int i = 0; i < iters; ++i) {
        f();
    }

    auto t1 = clock::now();
    std::chrono::duration<double, std::milli> ms = t1 - t0;

    return ms.count() / iters;
}


static void sink_checksum(const uint8_t* data, int n) {
    uint64_t s = 0;
    for (int i = 0; i < n; ++i) {
        s += data[i];
    }
    std::cout << "checksum: " << s << std::endl;
}


int main() {
    std::cout << "AVX2 supported: " << (__builtin_cpu_supports("avx2") ? "yes" :
"no") << std::endl;

    std::ifstream f("img.raw", std::ios::binary);
    f.read(reinterpret_cast<char*>(src), N);

    std::vector<uint8_t> out_scalar(N), out_avx2(N);

    blur3x3_scalar(src, out_scalar.data(), W, H);
```

```
    blur3x3_avx2(src, out_avx2.data(), W, H);

    std::cout << "max_abs_diff(scalar, avx2) = " << max_abs_diff(out_scalar.data(),
out_avx2.data(), N) << std::endl;

    constexpr int iters = 300;

    double t_scalar = bench_ms([&] { blur3x3_scalar(src, out_scalar.data(), W,
H); }, iters);
    double t_avx2   = bench_ms([&] { blur3x3_avx2(src, out_avx2.data(), W, H); },
iters);

    std::cout << "avg scalar: " << t_scalar << " ms" << std::endl;
    std::cout << "avg avx2  : " << t_avx2 << " ms" << std::endl;
    std::cout << "speedup   : " << (t_scalar / t_avx2) << "x" << std::endl;

    sink_checksum(out_avx2.data(), N);
    return 0;
}
```

Результаты

Вывод программы:

- `AVX2 supported: yes` — процессор поддерживает расширение AVX2, следовательно, оно может быть использовано для SIMD-оптимизации;

- `max_abs_diff(scalar, avx2)=0` — максимальное абсолютное отличие между выходными изображениями, полученными скалярной и SIMD-реализациями, равно 0, значит, результаты полностью совпадают и SIMD-версия корректна, что соответствует требованию проверки правильности путём сопоставления результатов;

- `checksum: 39692269` — контрольная сумма выходного массива (используется для фиксации результата и предотвращения исключения вычислений компилятором).

Время выполнения измерено для двух реализаций (усреднение по серии запусков):

- Среднее время скалярной версии: `avg scalar = **0.784474** ms`
- Среднее время SIMD-версии (AVX2): `avg avx2 = **0.204288** ms`

Полученное ускорение: выигрыш в **3.84** раза.

То есть SIMD-оптимизированная реализация работает примерно в **3.84 раза быстрее** по сравнению с исходной высокоуровневой реализацией. Это согласуется с принципом SIMD (параллельная обработка нескольких элементов данных одной инструкцией), на котором основаны векторные расширения x86.

Выводы

В ходе лабораторной работы была изучена практическая SIMD-оптимизация свёрточных операторов обработки изображений на процессорах x86 и реализована обработка изображения в соответствии с индивидуальным вариантом №7 — оператор размытия порядка 3 с маской 3×3 из единиц и коэффициентом нормировки d=1/9.

Корректность SIMD-реализации подтверждена сопоставлением результатов с эталонной скалярной версией: максимальное абсолютное отклонение между выходными массивами составило 0, т.е. результаты полностью совпали.

Экспериментальные измерения времени выполнения показали, что SIMD-оптимизированная версия работает значительно быстрее исходной: среднее время выполнения скалярной реализации составило 0.784474 мс, а AVX2-реализации — 0.204288 мс, что соответствует ускорению примерно в 3.84 раза.